Assignment 2: Model Development, Deployment and Monitoring

1. Project Objective

Build, evaluate and deploy the system from your project proposal using the dataset frozen in Assignment 1. Track experiments, compare baselines, test on unseen data, and deliver a local Docker service with a minimal interface, monitoring and retraining. Each group presents its work in a 20-minute presentation and live demo, followed by 10 minutes of questions.

Core Technical Pipeline:

- **Parts A–B:** Model development, baselines, experiment tracking and tuning.
- **Part C:** Evaluation, confidence intervals, slice analysis and errors.
- **Parts D–E:** Local Docker deployment, user interface, monitoring and retraining.
- **Parts F–G:** Reproducible submission, report and presentation.

Deadline

Submission: 11:59 PM, 15 November 2026

Instructor: Dr. Yogesh K. Meena

27 October 2026 • Semester I (2026-2027) • CS 203 Software Tools and Techniques in AI (Course Project/Assignment 2)

© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.

1. Overview and Rules

This assignment applies to **Projects 1, 4 and 5**. Start from repository tag `a1-final`. Follow Parts A–G and the project-specific requirements in Section 3.

- **Fixed splits:** train on training data and choose models, prompts, settings and thresholds using validation data only. Reserve test and gold test data for final evaluation.
- **Gold test protection:** keep its checksum unchanged from `HANDOFF_A2.md`. Log every evaluation on it, including each Part E gate check, in `TEST_SET_LOG.md`, with date, model version and reason. Never use it for training or tuning.
- **Data changes:** version fixes and additions with DVC after `a1-final`; explain them in `DATA_CHANGES.md`. Do not edit the gold test set.
- **Local inference:** all models and tools must run inside the container without external AI APIs, API keys or runtime downloads. Assignment 1 AI-generated data may be reused if versioned with its exact prompts in `prompts/`.

Part	Requirement	Marks
A	Model development and baselines	15
B	Experiment tracking and tuning	10
C	Evaluation and error analysis	15
D	Docker deployment and user interface	15
E	Monitoring, drift and retraining	15
F	Report, artefacts and reproducibility	10
G	Presentation and questions	20
	Total	**100**

2. Technical Requirements

Part A: Model Development and Baselines

Implement **at least two baselines**, including one trivial or unadapted method, and your proposed system. Explain changes from the proposal. Use config-driven training and record configurations, DVC hashes, seeds, hardware and training time. Compare validation performance with and without augmented and actively selected Assignment 1 data. Free compute tiers are sufficient.

Part B: Experiment Tracking and Tuning

Log every training/evaluation run in MLflow or Weights & Biases: parameters, metrics, data version, code commit and outputs. Run a systematic search over **at least two hyperparameters or configuration settings and 15 runs**, selecting on validation data only. Include a search-analysis plot and justify the quality/latency/size trade-off. Register the final model or configuration with versioned staging/production aliases. Each reported result must link to a run ID.

Part C: Evaluation and Error Analysis

- Use one evaluation script accepting a registered version and split. Compare all baselines and the final system on validation, test, gold test and shifted data, with **95% bootstrap confidence intervals**. Support improvement claims with an appropriate paired test.
- Report project-specific metrics and slices (Section 3), identify weak groups, and categorise **at least 20 test errors** with suggested fixes.
- Test robustness on Assignment 1 Part D augmented test data. Write `MODEL_CARD.md` covering intended use, data, metrics/intervals, slices, limitations and risks.

Part D: Docker Deployment and User Interface

- Submit a self-contained image for **linux/amd64, CPU only, at most 16 GB RAM**. Bundle all models, code, UI and gold test data. State image size and RAM requirements.
- Build from `code/Dockerfile` at `a2-final` without manual steps, using a pinned base image. Record the git commit in the image and display it in the UI.
- Document one command to start the service/UI and another to validate the full gold set offline, e.g. `docker run --network none <image> validate`. Print metrics and save JSON matching the report within a stated tolerance.
- Offline operation is checked in two ways: `validate` runs with `--network none`, and the UI is started with only its port published (`-p`) on a machine with the internet disconnected. The code must make no outbound calls.
- Provide a minimal Gradio/Streamlit UI with user input, bundled examples, output, confidence, version, latency, monitoring and validation results beside reported values. UI validation may use a documented fixed subset if full CPU evaluation is slow.
- Provide REST prediction, health and version/commit endpoints. Log requests/responses, version, latency and confidence without personal information. Measure CPU p50/p95 latency, throughput and memory; state hardware.
- On each push, GitHub Actions must run unit tests, sample Assignment 1 validation, an image build and an offline smoke request (`--network none`).

Part E: Monitoring, Drift and Retraining

- Replay traffic that gradually shifts to the Assignment 1 shifted split, using a fixed seed/schedule. Track **two label-free signals and one labelled quality metric**.
- Choose drift tests/thresholds on validation data. Report detection delay and false alarms; show whether signals track quality using a plot or correlation.
- After an alert, a one-command pipeline must select data with the Assignment 1 active-learning method, reveal its labels from a pre-labelled pool (simulated annotator), validate, retrain and evaluate.
- Promote only if shifted-data performance improves and gold-test performance stays within a tolerance fixed in advance. Keep evaluation data out of retraining. Log gate checks without tuning on gold results. Show a promotion, any rejection and rollback.

3. Project-Specific Requirements

These requirements add to Parts A–E.

Project 1: AgentEvalOps

Build/tune: use a local, quantised open-weight LLM of **at most 8B parameters** and local tools/retrieval; rerun any API-based baselines locally. Compare direct answers without tools and the original function-calling loop. Improve planning, tool descriptions, retries/error feedback or loop/step guards. Version the model, prompts and configuration. Search prompt version, temperature, step limit, retries or model on validation template families, under a token/latency budget. Log MLflow traces.

Evaluate: task success, Assignment 1 failure categories, steps, tokens and latency. Use Assignment 1 failure detectors; validate any LLM judge against human labels using Cohen's kappa. Slice by template family, ambiguous/unanswerable tasks and fault type.

Deploy/monitor: accept text tasks; return answers and trace IDs. Load prompts/configurations from the bundled registry. Show steps, tool calls, arguments and outputs. Monitor loops, malformed calls, tool errors, steps, latency and tokens. Inject faults, change tool formats or swap model versions to demonstrate drift. Retune prompts/configurations on new labelled trajectories and gate on held-out template families.

Project 4: Course Advisory SLM

Build/tune: compare zero-shot, few-shot and curriculum-JSON retrieval baselines. The rule engine is the reference. Fine-tune an SLM of **at most 3B parameters** using LoRA/QLoRA; check answers against the rule engine before display. Search rank, learning rate, epochs and data mix.

Evaluate: set-level precision/recall/F1 for remaining core and eligible courses, exact match, constraint violations and invented codes (measured on raw model output, before the rule-engine check). Two annotators rate **at least 50 test justifications**; report kappa. Slice by programme, query type and transcript length.

Deploy/monitor: serve the quantised model, curriculum and rule engine locally; report CPU latency. Accept completed courses/questions; show answer, justification and rule-check result. Monitor unknown codes, rule rejections, query embedding drift or answer length. Demonstrate drift with real Google Forms queries and a curriculum update; regenerate affected data, validate, fine-tune and gate.

Project 5: Voice Helpdesk ASR

Build/tune: baseline with the Assignment 1 Whisper version and a text-only intent classifier on its transcripts. Fine-tune an ASR model of Whisper small or base size on training speakers and add intent routing. Search learning rate, frozen layers, augmentation or decoding settings.

Evaluate: transcript WER/CER and intent accuracy/macro-F1; compare intent accuracy on ASR and human transcripts. Slice by language mix, noise, device, scripted/spontaneous speech and speaker region. Report Cohort B separately.

Deploy/monitor: accept uploaded/microphone audio; return transcript, intent and confidence. Bundle consented clips; report CPU real-time factor. Monitor decoder confidence, language distribution, audio statistics or unseen words. Shift to Cohort B, select low-confidence clips for transcription, retrain and compare Cohort B/gold WER before and after.

4. Reproducibility and Submission (Part F)

README: reproduce final training/evaluation from `a2-final` in **at most five commands**; separately document Docker load/build, startup and validation with expected output. Update `requirements.txt`, `requirements_freeze.txt` and `ENVIRONMENT.md`. Record software/hardware versions, pretrained-model sources/revisions, run IDs, registry versions, weight checksums and all seeds. Use relative paths; remove temporary files and secrets from code/history.

Submit `CS203_A2_Project<N>_<roll-numbers>.zip` by **11:59 PM, 15 November 2026**. Upload to Drive/OneDrive with link-download access and submit the link on Google Classroom. No earlier checkpoint; do not change files after the deadline. Include:

- `code/`: repository at `a2-final`, including `.git`, GitHub URL, all code/configs, DVC files, Docker/CI files, required documentation and exact AI prompts in `prompts/`.
- `data/`: all final splits, additions, labels, replay and retraining data, matching DVC. Anonymise personal information.
- `artefacts/`: final/retrained models, run/registry/dashboard exports and `docker_image.tar.gz`. Include `DOCKER.md` with SHA-256, load/run/validation commands and expected output. Oversized images/weights may be separate downloads with links/checksums in `DOCKER.md/MODELS.md`.
- `supporting_files/`: predictions, evaluation, search and load-test outputs; monitoring, drift, retraining and gate logs; error sheets and plot data; a **three-minute recording** of the service and complete retraining loop.
- `report/`: `report.pdf`, **at most eight pages plus appendices**, covering Parts A–F and member contributions. Append an evidence index linking results/findings to files/run IDs, and describe AI assistance.
- `presentation/`: `slides.pdf`, exactly as presented.

Before submission, unzip into a clean folder and reproduce the results. On another CPU-only machine, load the image, verify offline operation, open the UI using the documented local setup, and validate. The Dockerfile must rebuild the working image.

5. Presentation (Part G)

The presentation is mandatory: 20 minutes including a live demo, then 10 minutes of questions. Dates will be announced later on Google Classroom. Every member must present and explain any submitted code/result; answers to questions are marked individually. Show inputs, validation, monitoring, a drift alert and retraining; a recording of slow retraining is acceptable. Present only the submitted system and numbers.

6. Evaluation and Verification

AI-assisted checks examine code/model/data consistency, local inference, prompts, gold-set protection and evidence. TAs load/rebuild the image, test offline CPU operation and the UI, reproduce metrics and check requirement coverage. Final marking considers AI findings.

- Unsupported or irreproducible results receive no credit. State tolerances and retain supporting files/run IDs.

- A failed container, GPU/internet/API dependency or mismatched validation loses Part D marks; affected Part C results are unsupported. Gold-set tuning or checksum changes invalidate affected results.
- Preserve original prompts/logs/exports. Include originals, corrections and correction scripts when changes are necessary.

Git history, the contribution table and answers determine individual adjustments where contributions differ substantially. Commit your own work. The Institute Honour Code applies: `https://iitgn.ac.in/students/honourcode`.

© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.